

Complementi di Matematica e Calcolo Numerico A.A. 2023-2024

Laboratorio 4

Programmare con Matlab: Script-files

Che cos' è uno *script file*?

- È un file con estensione .m (ad esempio: `myfile.m`).
- Contiene una sequenza di istruzioni Matlab, scritte come se fossero digitate in modalità interattiva.
- Digitando il nome di uno *script-file* a destra del prompt:

```
>> myfile
```

vengono eseguite in successione tutte le istruzioni contenute nel file.

- Le variabili assegnate in uno *script-file* sono visibili dall'esterno, ovvero persistono in memoria al termine dell'esecuzione.

Alcune buone regole

- Il nome di uno *script-file* deve essere diverso dai nomi delle variabili che esso elabora e dai nomi delle variabili presenti in Workspace, altrimenti non verrà eseguito.
- Non assegnare ad uno *script-file* il nome di una funzione predefinita di Matlab. Per verificare se un nome esiste già:

```
>> exist('nome')
```

Matlab come linguaggio di programmazione

Come ogni linguaggio di programmazione Matlab fornisce gli operatori relazionali

$==, \sim, <, >, <=, >=$

Per convenzione si associa il valore 1 a “vero” e 0 a “falso”.

$a == b \longrightarrow 1$ se $a = b$ 0 altrimenti

$a \sim b \longrightarrow 1$ se $a \neq b$ 0 altrimenti

Questi operatori si applicano anche a vettori, ad esempio

```
>> x = [4,1,19]; y = [5,0,3];
```

```
>> b = x <= y
```

```
>> b =
```

```
1 x 3 logical array
```

```
1    0    0
```

Si osservi che sebbene sia costituito di 0 e di 1, b è definito come un vettore logico e non semplicemente come un vettore di numeri reali 0 ed 1.

Come ogni linguaggio di programmazione Matlab fornisce costrutti appositi per realizzare istruzioni condizionate o cicli.

Se si vuole eseguire una o più operazioni solo nel caso in cui sia verificata una data condizione si può utilizzare il costrutto `if` che ha la seguente sintassi

```
if (condizione==true)
    istruzioni
end
```

ad esempio potrei controllare se la variabile x sia non negativa prima di calcolarne la radice quadrata

```
>> if(x>=0)
        r=sqrt(x);
>> end
```

Se invece vogliamo ripetere un numero fissato di volte alcune istruzioni possiamo realizzare un ciclo con il comando Matlab `for` che ha la seguente sintassi

```
for contatore=inizio:passo:fine
    istruzioni
end
```

Esempio: Dato un numero naturale positivo n vogliamo calcolare la matrice triangolare superiore A di ordine n definita da

$$A_{i,j} = \begin{cases} n & \text{se } i = j \\ i/(j+1) & \text{se } i < j \end{cases}$$

Ad esempio se $n = 4$

$$A = \begin{bmatrix} 4 & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \\ 0 & 4 & \frac{2}{4} & \frac{2}{5} \\ 0 & 0 & 4 & \frac{3}{5} \\ 0 & 0 & 0 & 4 \end{bmatrix}$$

Per calcolare coefficienti di A si potrebbe realizzare un doppio ciclo

```
for i = 1:n
    for j=i+1:n
        A(i,j)=i/(j+1);
    end
    A(i,i)=n;
end
```

ma in Matlab è buona regola prima di scrivere un ciclo vedere se è possibile evitarlo tramite un opportuno uso di istruzioni vettoriali. Scriviamo uno *script-file* che letta da tastiera la dimensione calcoli la matrice triangolare superiore A nel modo seguente.

Per generare un nuovo *script-file* selezioniamo dal menu:

file $\longrightarrow$ new $\longrightarrow$ Mfile

mat_trisup.m

```
n=input('inserisci dimensione matrice ');
if(n<0)
    disp('dimensione della matrice inaccettabile');
    return;
end
A=eye(n)*n;
for i=1:n
    A(i,i+1:n)=i./[i+2:n+1];
end
A
```

Per eseguire il codice è sufficiente digitare **mat_trisup** a destra del prompt nella finestra di comando. Scegliendo come dimensione 4 otteniamo:

```
>> mat_trisup
inserisci dimensione matrice 4
A =
    4.00000    0.33333    0.25000    0.20000
    0.00000    4.00000    0.50000    0.40000
    0.00000    0.00000    4.00000    0.60000
    0.00000    0.00000    0.00000    4.00000
```

Metodi diretti per sistemi lineari

Siano $A \in \mathbb{R}^{n \times n}$ una matrice quadrata non singolare ($\det(A) \neq 0$) e $b \in \mathbb{R}^n$ un vettore assegnati, allora esiste un unico vettore $x \in \mathbb{R}^n$ che risolve il sistema lineare

$$Ax = b$$

In Matlab, una volta assegnati A e b , si può calcolare la soluzione x del sistema con l'operatore `\`, che per una matrice quadrata generica implementa il metodo di **Eliminazione Gaussiana**, per una triangolare inferiore la **sostituzione in avanti** e per una triangolare superiore la **sostituzione all'indietro**:

```
>> x = A\b
```

Esercizio 1

Risolvere il seguente sistema lineare con l'operatore `\`:

$$\begin{bmatrix} -16 & 8 & -7 \\ 1 & -5 & -3 \\ 1 & 10 & 14 \end{bmatrix} x = \begin{bmatrix} -9 \\ 4 \\ -13 \end{bmatrix} \longrightarrow x = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}$$

Visualizzare i risultati in **format long e** e confrontarli con quelli che si ottengono dal prodotto dell'inversa di A con il vettore b ossia

```
>> x = inv(A)*b
```

Perchè i risultati non coincidono? Perchè non si usa questa seconda strategia?

Esercizio 2 (E se la matrice fosse singolare?)

Il teorema di Rouché-Capelli ci dice che il sistema lineare $Ax = b$ ammette soluzione se e solo se $\text{rk}(A) = \text{rk}(A|b)$.

Dopo aver calcolato il determinante e il rango della matrice del sistema e il rango della matrice orlata risolvere con `\` i seguenti sistemi lineari, facendo molta attenzione ai messaggi d'errore o warning. Nel secondo e terzo caso confrontare il valore del termine noto con quello del prodotto matrice-soluzione.

$$\begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{bmatrix} x = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix},$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 5 & 7 & 9 \end{bmatrix} x = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix},$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 5 & 7 & 9 \end{bmatrix} x = \begin{bmatrix} 5 \\ 2 \\ 3 \end{bmatrix}.$$

Si osserva che il determinante delle matrici è nullo. Il primo sistema non ha soluzioni e il comando `\` lo segnala con un messaggio di errore, il secondo sistema ammette infinite soluzioni e matlab ne individua una, il terzo sistema ancora non ha soluzione ma il comando `\` non segnala un errore bensì determina un vettore che approssima la soluzione del sistema “nel senso dei minimi quadrati” e fornisce quindi qualcosa di diverso senza segnalare un errore.

Esercizio 3 (Sistemi di "grandi" dimensioni)

Costruire la matrice A $N \times N$ con $N = 43$ avente come soli elementi non nulli quelli della diagonale principale uguali a 7, quelli della prima riga eccetto $A(1, 1)$ uguali a 0.1 e quelli della seconda sottodiagonale uguali a -3 . Sia b il vettore colonna unitario di lunghezza N . Risolvere il sistema lineare $Ax = b$ mediante il metodo di eliminazione gaussiana.

FATTORIZZAZIONE LU

Quando è possibile fattorizzare la matrice dei coefficienti di un sistema lineare A nel prodotto $A = LU$, con L matrice triangolare inferiore (con tutti i coefficienti uguali a 1 sulla diagonale) ed U triangolare superiore, per risolvere un sistema lineare $Ax = b$ possiamo procedere come segue: una volta determinati L ed U si riscrive il sistema lineare equivalente

$$LUx = b$$

(ricordiamo che due sistemi lineari sono equivalenti se hanno la stessa soluzione). Ponendo allora $y = Ux$ è possibile determinare la soluzione x del sistema considerato risolvendo in sequenza per sostituzioni i due sistemi triangolari:

$$\begin{aligned} Ly &= b \\ Ux &= y \end{aligned}$$

Quando A è fattorizzabile nel prodotto $A = LU$, il metodo di eliminazione Gaussiana (e quindi \) calcola tale fattorizzazione e

contestualmente risolve per sostituzione i due sistemi triangolari con matrici dei coefficienti L ed U fino ad ottenere la soluzione del sistema lineare dato $Ax = b$.

Quando non esiste la fattorizzazione $A = LU$ combinando l'eliminazione Gaussiana con scambi di righe (tecnica detta del pivoting parziale) è possibile calcolare una matrice triangolare superiore U , una triangolare inferiore L (con coefficienti diagonali uguali a 1) e una matrice di permutazione P (una matrice che moltiplicata a sinistra di un'altra ne scambia le righe) tali che

$$PA = LU.$$

In questo caso la soluzione del sistema lineare $Ax = b$, equivalente a $PAx = Pb$, si calcolerà risolvendo in sequenza i due sistemi triangolari

$$Ly = Pb$$

$$Ux = y$$

Il comando `lu` di Matlab calcola la fattorizzazione LU di A o quando serve la fattorizzazione LU di PA . La sua sintassi è:

`[L,U,P]= lu(A)`

Se non vengono effettuati scambi di righe la matrice P sarà semplicemente la matrice identità.

Esercizio 4. Si consideri la matrice:

$$A = \begin{bmatrix} 4 & -2 & 1 \\ -2 & 5 & -2 \\ 1 & -2 & 6 \end{bmatrix}.$$

- si calcoli la fattorizzazione LU tramite il comando Matlab `lu` usando la sintassi

```
>> [L,U,P]=lu(A)
```

- si osservi se la matrice di permutazione P fornita da Matlab è o meno l'identità.
- Si scelga $b = A * \mathbf{ones}(3, 1)$, e si consideri il sistema lineare $Ax = b$ che avrà in tal modo soluzione esatta nota $x = \mathbf{ones}(3, 1)$. Risolvere il sistema lineare assegnato sfruttando la fattorizzazione calcolata.

Esercizio 5 Si consideri la seguente matrice:

$$A = \begin{bmatrix} -5 & 8 & -7 \\ 12 & -5 & -3 \\ 1 & 10 & 14 \end{bmatrix}.$$

Si ripeta quanto richiesto per l'esercizio precedente. Si osservi che la matrice di permutazione P non è l'identità, il che significa che è stato effettuato il pivoting. Pertanto abbiamo $PA = LU$.

Quando dobbiamo risolvere più sistemi lineari con diversi termini noti, ma tutti di ugual matrice A , è più conveniente separare il calcolo della fattorizzazione da quello della soluzione del sistema.

Esercizio 6 Sia A la matrice 100×100 tridiagonale avente coefficienti uguali a 4 sulla diagonale principale e uguali a -2 sulle prime sopra e sotto diagonali. Si calcoli l'inversa A^{-1} resolvendo i sistemi lineari $Ax = e_i$, dove $e_i = (0, \dots, \overbrace{1}^i, \dots, 0)^T$ $i = 1, \dots, n$ denotano i vettori della base canonica di $\mathbb{R}^n$. La soluzione dell' i -esimo sistema $Ax = e_i$, fornisce infatti la colonna i -esima della matrice A^{-1} . Si proceda nei due modi seguenti confrontandone le prestazioni mediante i comandi `tic` e `toc` per il calcolo dei tempi di calcolo.

- Si risolva ogni sistema per eliminazione Gaussiana con il comando `\`
- Poichè la matrice dei coefficienti di ciascun sistema è sempre A si calcoli una sola volta la fattorizzazione LU e si risolvano i diversi sistemi per sostituzioni in avanti e indietro.

IL FENOMENO DEL *FILL-IN*

Quando una matrice A presenta tanti coefficienti uguali a zero (si parla anche di matrice sparsa) ci si potrebbe chiedere se nelle stesse posizioni in cui ci sono zeri in A ci saranno certamente zeri anche nelle corrispondenti posizioni delle matrici L ed U della fattorizzazione $A = LU$. Se ciò fosse vero potremmo risparmiare costi computazionali evitando di calcolare tali componenti di L e di U perchè già note. Purtroppo la risposta a questa domanda in generale è negativa, e quello che accade viene chiamato fenomeno del “fill-in” per il fatto che nelle posizioni in cui A aveva zeri si ha un “rimpimento” con coefficienti diversi da zero nelle matrici della fattorizzazione. Fanno eccezione le matrici a banda ovvero quelle matrici che hanno solo gli elementi di un certo numero di sopra o sotto diagonali diversi da zero. Il comando `spy(A)` di matlab vi mostra con una rappresentazione grafica le posizioni nella matrice A in cui i coefficienti non sono nulli.

Esercizio 7

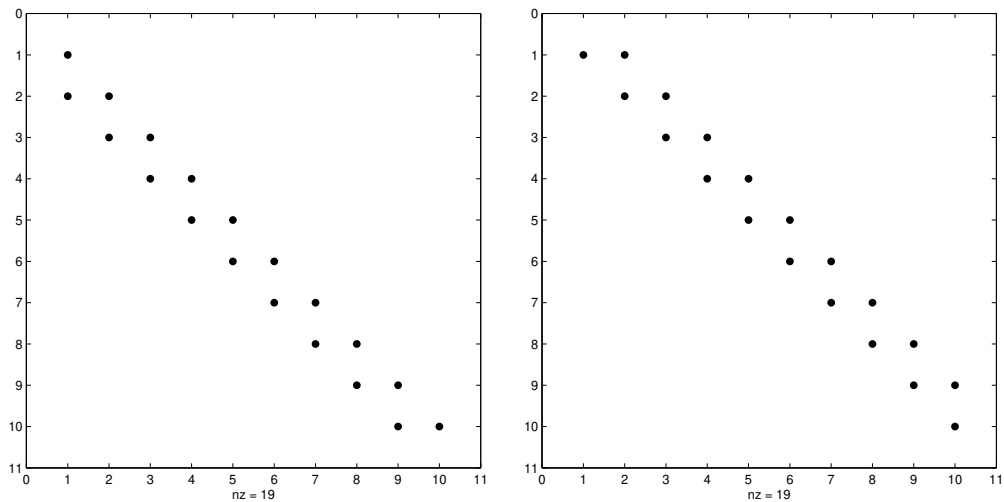
Per ognuna delle seguenti matrici calcolare la fattorizzazione LU , controllare se viene effettuato il pivoting e verificare il fenomeno del *fill-in* applicando il comando *spy* ad L e U :

$$A_{10 \times 10} = \begin{bmatrix} 4 & -1 & & & \\ -1 & 4 & -1 & & \\ & -1 & 4 & -1 & \\ \dots & \dots & \dots & \dots & \dots \\ & & & -1 & 4 \end{bmatrix} \quad B_{10 \times 10} = \begin{bmatrix} 3 & -2 & & & \\ -2 & 3 & -2 & & \\ & -2 & 3 & -2 & \\ \dots & \dots & \dots & \dots & \dots \\ & & & -2 & 3 \end{bmatrix}$$

$$C_{10 \times 10} = \begin{bmatrix} 2 & 0.1 & 0.1 & \dots & 0.1 \\ 0.1 & 2 & & & \\ 0.1 & & 2 & & \\ \dots & & & \dots & \\ 0.1 & & & & 2 \end{bmatrix} \quad D_{7 \times 7} = \begin{bmatrix} 1 & & & & 1 \\ & 1 & & & 1 \\ & & 1 & & 1 \\ & & & \dots & \dots \\ 2 & 3 & 4 & \dots & 8 \end{bmatrix}$$

ed esempio

```
>> A=4*diag(ones(10,1))-diag(ones(9,1),1)-diag(ones(9,1),-1)
>> [L U P]=lu(A)
>> figure(1)
>> spy(L)
>> figure(2)
>> spy(U)
```



Osserviamo che

- la matrice A è a banda, non viene effettuato il pivoting e quindi le matrici L ed U mantengono la struttura a banda.
- la matrice B è a banda, viene però effettuato il pivoting e quindi le matrici L ed U perdono la struttura a banda.
- le matrici C e D sono sparse, ma le matrici L ed U sono piene.

FATTORIZZAZIONE DI *CHOLSKY*

Se $A \in \mathbb{R}^{n \times n}$ è una matrice simmetrica definita positiva, allora esiste una matrice $R \in \mathbb{R}^{n \times n}$ triangolare superiore tale che

$$A = R^T R.$$

Tale fattorizzazione è detta fattorizzazione di **Cholesky**.

Il comando `R = chol(A)` di Matlab determina tale fattorizzazione.

Esempio

La matrice

$$A = \begin{bmatrix} 1 & -1 & 0 \\ -1 & 5 & 6 \\ 0 & 6 & 10 \end{bmatrix},$$

è simmetrica definita positiva. Calcolare con il comando `chol` di Matlab la fattorizzazione di Cholesky di A . Si risolva il sistema lineare $Ax = b$ con `b=[3;1;1]` sfruttando la fattorizzazione calcolata. Osserviamo che poichè $A = R^T R$, la risoluzione del sistema lineare di partenza comporta la risoluzione in sequenza dei due sistemi triangolari

$$R^T y = b$$

$$Rx = y$$

```
A=[ 1  -1   0; -1  5 6; 0 6 10];
```

```
R=chol(A);
```

```
b=[3;1;1];
```

```
y=R'\b;
```

```
x=R\y
```